

```

import streamlit as st
from groq import Groq
from datetime import datetime, timedelta
import time
import base64
import json
import os
import re

# --- PAGE CONFIGURATION ---
st.set_page_config(
    page_title="VectorWare AI",
    page_icon="🌀",
    layout="centered",
    initial_sidebar_state="expanded"
)

# --- THEME STATE ---
if "dark_mode" not in st.session_state:
    st.session_state.dark_mode = True

# --- THEME COLORS ---
if st.session_state.dark_mode:
    BG = "radial-gradient(circle at top center, #150a3a 0%, #05030d 100%)"
    SIDEBAR_BG = "#090615"
    CARD_BG = "rgba(147, 85, 255, 0.06)"
    TEXT = "#f3f4f6"
    SUBTEXT = "#9ca3af"
    INPUT_BG = "#160f33"
    MSG_TEXT = "#e2e8f0"
    BORDER = "rgba(147, 85, 255, 0.2)"
else:
    BG = "radial-gradient(circle at top center, #f0ebff 0%, #ffffff 100%)"
    SIDEBAR_BG = "#f5f0ff"
    CARD_BG = "rgba(147, 85, 255, 0.08)"
    TEXT = "#1a1a2e"
    SUBTEXT = "#6b7280"
    INPUT_BG = "#ffffff"
    MSG_TEXT = "#1f2937"
    BORDER = "rgba(147, 85, 255, 0.3)"

st.markdown(f"""
    <style>
    @import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;

```

```

.stApp, .stApp p, .stApp h1, .stApp h2, .stApp h3, .stApp button, .stApp text
  font-family: 'Inter', sans-serif !important;
}}

[data-testid="stChatMessage"] [data-testid="stChatMessageAvatarUser"] div,
[data-testid="stChatMessage"] [data-testid="stChatMessageAvatarAssistant"] di
  font-size: 0px !important;
  color: transparent !important;
}}
[data-testid="stChatMessageAvatarUser"] div::before {{
  content: "👤";
  font-size: 1.2rem !important;
}}
[data-testid="stChatMessageAvatarAssistant"] div::before {{
  content: "🌀";
  font-size: 1.2rem !important;
}}

.stApp {{
  background: {BG};
  color: {TEXT} !important;
}}

section[data-testid="stSidebar"] {{
  background: {SIDEBAR_BG} !important;
  border-right: 1px solid {BORDER} !important;
}}

section[data-testid="stSidebar"] * {{
  color: {TEXT} !important;
}}

.stChatInputContainer {{
  background: transparent !important;
  padding: 0 !important;
}}

.stChatInput input {{
  background-color: {INPUT_BG} !important;
  color: {TEXT} !important;
  border: 1px solid rgba(147, 85, 255, 0.3) !important;
  border-radius: 24px !important;
  padding: 12px 20px !important;
}}

.stChatInput input:focus {{

```

```

        border-color: rgba(147, 85, 255, 0.7) !important;
        box-shadow: 0 0 15px rgba(147, 85, 255, 0.3) !important;
    }}

[data-testid="stChatMessageContent"] p {{
    color: {MSG_TEXT} !important;
    line-height: 1.75 !important;
}}

[data-testid="stChatMessageContent"] strong {{
    color: #d8b4fe !important;
}}

@keyframes spiralGlow {{
    0%, 100% {{ filter: drop-shadow(0 0 6px #9b59ff) drop-shadow(0 0 12px #9b
    50% {{ filter: drop-shadow(0 0 16px #00d4ff) drop-shadow(0 0 25px #9b59ff
}}
@keyframes spiralSpin {{
    0% {{ transform: rotate(0deg); }}
    100% {{ transform: rotate(360deg); }}
}}

.spiral-idle {{
    animation: spiralGlow 4s ease-in-out infinite;
    display: inline-block;
}}

.spiral-thinking {{
    animation: spiralSpin 1.2s linear infinite, spiralGlow 1.5s ease-in-out i
    display: inline-block;
}}

.greeting {{
    font-size: 2.4rem;
    font-weight: 300;
    color: {TEXT} !important;
    margin: 0;
}}

.greeting-sub {{
    font-size: 2.4rem;
    font-weight: 600;
    background: linear-gradient(90deg, #9b59ff, #bf00ff, #00d4ff);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    margin: 0 0 15px 0;
}}

.about-section {{

```

```

        background: {CARD_BG};
        border: 1px solid {BORDER};
        border-radius: 14px;
        padding: 14px;
        margin-top: 15px;
    }}

    .founder-tag {{
        background: rgba(147, 85, 255, 0.15);
        border-radius: 8px;
        padding: 6px 10px;
        margin: 5px 0;
        font-size: 0.8rem;
        color: #d8b4fe !important;
        display: block;
    }}

    .stButton button {{
        background: rgba(147, 85, 255, 0.1) !important;
        color: {TEXT} !important;
        border: 1px solid rgba(147, 85, 255, 0.25) !important;
        border-radius: 12px !important;
        transition: all 0.2s !important;
    }}

    .stButton button:hover {{
        background: rgba(147, 85, 255, 0.25) !important;
        border-color: rgba(147, 85, 255, 0.6) !important;
    }}

    .upload-zone {{
        border: 1px dashed rgba(147, 85, 255, 0.4);
        border-radius: 14px;
        padding: 12px 16px;
        background: rgba(147, 85, 255, 0.04);
        margin-bottom: 10px;
    }}

    .upload-label {{
        color: #9b59ff;
        font-size: 0.78rem;
        font-weight: 500;
        margin-bottom: 4px;
        display: block;
    }}

    [data-testid="stFileUploader"] {{
        background: transparent !important;
        border: none !important;
        padding: 0 !important;
    }}

```

```

}}
[data-testid="stFileUploader"] section {{
    background: transparent !important;
    border: none !important;
    padding: 0 !important;
}}

.img-preview-pill {{
    display: inline-flex;
    align-items: center;
    gap: 6px;
    background: rgba(147, 85, 255, 0.15);
    border: 1px solid rgba(147, 85, 255, 0.3);
    border-radius: 20px;
    padding: 4px 10px;
    font-size: 0.78rem;
    color: #d8b4fe;
    margin-bottom: 8px;
}}

.name-card {{
    background: {CARD_BG};
    border: 1px solid rgba(147, 85, 255, 0.25);
    border-radius: 20px;
    padding: 32px 28px;
    margin: 20px 0;
    text-align: center;
}}

.history-label {{
    color: #6b7280;
    font-size: 0.68rem;
    font-weight: 700;
    letter-spacing: 0.08em;
    margin: 12px 0 4px 0;
    padding-left: 2px;
}}

.name-badge {{
    background: rgba(147, 85, 255, 0.12);
    border: 1px solid rgba(147, 85, 255, 0.25);
    border-radius: 10px;
    padding: 7px 12px;
    margin-bottom: 10px;
    font-size: 0.8rem;
    color: #d8b4fe;
    text-align: center;
}}

```

```
.thinking-box {{
  background: rgba(147, 85, 255, 0.06);
  border: 1px solid rgba(147, 85, 255, 0.2);
  border-radius: 12px;
  padding: 10px 14px;
  margin-bottom: 10px;
  font-size: 0.82rem;
  color: #a78bfa;
}}
```

```
.memory-pill {{
  display: inline-flex;
  align-items: center;
  gap: 5px;
  background: rgba(0, 212, 255, 0.08);
  border: 1px solid rgba(0, 212, 255, 0.2);
  border-radius: 20px;
  padding: 3px 10px;
  font-size: 0.72rem;
  color: #00d4ff;
  margin: 2px;
}}
```

```
.copy-btn {{
  background: rgba(147, 85, 255, 0.1);
  border: 1px solid rgba(147, 85, 255, 0.2);
  border-radius: 8px;
  padding: 3px 10px;
  font-size: 0.72rem;
  color: #a78bfa;
  cursor: pointer;
  transition: all 0.2s;
}}
```

```
.copy-btn:hover {{
  background: rgba(147, 85, 255, 0.25);
}}
```

```
.search-pill {{
  display: inline-flex;
  align-items: center;
  gap: 5px;
  background: rgba(147, 85, 255, 0.08);
  border: 1px solid rgba(147, 85, 255, 0.2);
  border-radius: 20px;
  padding: 3px 10px;
  font-size: 0.72rem;
```

```

        color: #c084fc;
        margin-bottom: 8px;
    }}
</style>
""", unsafe_allow_html=True)

# --- STATE MANAGEMENT ---
if "messages" not in st.session_state:
    st.session_state.messages = []
if "chat_histories" not in st.session_state:
    st.session_state.chat_histories = []
if "pending_image" not in st.session_state:
    st.session_state.pending_image = None
if "user_name" not in st.session_state:
    st.session_state.user_name = None
if "memory_facts" not in st.session_state:
    st.session_state.memory_facts = []
if "voice_prompt" not in st.session_state:
    st.session_state.voice_prompt = None

# --- MEMORY FILE ---
MEMORY_FILE = "vectorware_memory.json"

def load_memory(name):
    if os.path.exists(MEMORY_FILE):
        with open(MEMORY_FILE, "r") as f:
            data = json.load(f)
            return data.get(name, {}).get("facts", [])
    return []

def save_memory(name, facts):
    data = {}
    if os.path.exists(MEMORY_FILE):
        with open(MEMORY_FILE, "r") as f:
            data = json.load(f)
    data[name] = {"facts": facts, "updated": datetime.now().isoformat()}
    with open(MEMORY_FILE, "w") as f:
        json.dump(data, f, indent=2)

def extract_facts(text, name):
    """Extract memorable facts from conversation"""
    facts = []
    patterns = [
        r"I(?:'m| am) (?:a |an )?(.+?) (?:\.|,|$)",
        r"I work (?:as |in |at )?(.+?) (?:\.|,|$)",
        r"I(?:'m| am) from (.*?) (?:\.|,|$)",
        r"I(?:'m| am) based in (.*?) (?:\.|,|$)",
    ]

```

```

        r"I love (.+?) (?:\.|,|$)",
        r"my (?:company|business|startup) (?:is |called )?(.+?) (?:\.|,|$)",
        r"I(?:'m| am) (\d+) years old",
        r"I speak (.+?) (?:\.|,|$)",
        r"I(?:'m| am) learning (.+?) (?:\.|,|$)",
    ]
    for pattern in patterns:
        matches = re.findall(pattern, text, re.IGNORECASE)
        for match in matches:
            fact = match.strip()
            if len(fact) > 3 and len(fact) < 80:
                facts.append(fact)
    return facts

# --- HELPERS ---
def encode_image(uploaded_file):
    bytes_data = uploaded_file.read()
    b64 = base64.b64encode(bytes_data).decode("utf-8")
    return b64, uploaded_file.type

def get_time_greeting():
    hour = datetime.now().hour
    if hour < 12:
        return "Good morning"
    elif hour < 17:
        return "Good afternoon"
    else:
        return "Good evening"

def group_histories_by_date(histories):
    now = datetime.now()
    today, yesterday, this_week, older = [], [], [], []
    for chat in histories:
        chat_date = chat.get("date", now)
        diff = (now - chat_date).days
        if diff == 0:
            today.append(chat)
        elif diff == 1:
            yesterday.append(chat)
        elif diff <= 7:
            this_week.append(chat)
        else:
            older.append(chat)
    return today, yesterday, this_week, older

def web_search(query):
    """Search the web using Tavily"""

```



```

try:
    from tavily import TavilyClient
    tavily = TavilyClient(api_key=st.secrets["TAVILY_API_KEY"])
    results = tavily.search(query=query, max_results=3)
    if results and results.get("results"):
        snippets = []
        for r in results["results"][:3]:
            snippets.append(f"• {r['title']}: {r['content'][:200]}")
        return "\n".join(snippets)
except Exception as e:
    return None
return None

def needs_web_search(prompt):
    """Detect if prompt needs live web data"""
    keywords = [
        "latest", "today", "current", "news", "price", "weather",
        "right now", "this week", "trending", "update", "live",
        "score", "stock", "crypto", "bitcoin", "2025", "2026",
        "recently", "just happened", "what happened"
    ]
    return any(kw in prompt.lower() for kw in keywords)

# --- API ---
client = Groq(api_key=st.secrets["GROQ_API_KEY"])

# =====
# NAME ONBOARDING — only shows once
# =====
if st.session_state.user_name is None:
    st.markdown("""
        <div style='padding: 40px 0 10px 0; text-align: center;'>
            <div class='spiral-idle' style='display: block; margin-bottom: 15px;'>
                <svg width="60" height="60" viewBox="0 0 200 200">
                    <defs>
                        <linearGradient id="spiralMain" x1="0%" y1="0%" x2="100%"
                            <stop offset="0%" style="stop-color: #bf00ff"/>
                            <stop offset="100%" style="stop-color: #7b2fff"/>
                        </linearGradient>
                    </defs>
                    <path d="M100,100 C100,60 140,40 160,70 C180,100 160,140 130,
                        C90,165 50,140 40,100 C25,50 60,15 100,15
                        C150,15 185,55 185,100 C185,155 140,190 90,185"
                        fill="none" stroke="url(#spiralMain)" stroke-width="18"
                        stroke-linecap="round"/>
                </svg>
            </div>
    """)

```

```

        </div>
"""", unsafe_allow_html=True)

st.markdown(f"""
    <div class='name-card'>
        <p style='font-size:1.6rem; font-weight:300; color:{TEXT}; margin:0;'>
        <p style='font-size:1.8rem; font-weight:700; background: linear-gradi
            -webkit-background-clip: text; -webkit-text-fill-color: tra
                VectorWare AI
        </p>
        <p style='color:#9ca3af; font-size:0.95rem; margin:0 0 24px 0;'>
            Next-generation AI – before we begin, what's your name?
        </p>
    </div>
"""", unsafe_allow_html=True)

c1, c2, c3 = st.columns([0.15, 0.7, 0.15])
with c2:
    name_input = st.text_input(
        label="name",
        placeholder="Enter your name...",
        label_visibility="collapsed",
        key="name_field"
    )
    if st.button("Let's Go →", use_container_width=True):
        if name_input.strip():
            st.session_state.user_name = name_input.strip()
            st.session_state.memory_facts = load_memory(name_input.strip())
            st.rerun()
        else:
            st.warning("Please enter your name to continue.")
st.stop()

# =====
# SIDEBAR
# =====
with st.sidebar:
    st.markdown("""
        <div style='text-align:center; padding: 20px 0 10px 0;'>
            <div class='spiral-idle'>
                <svg width="38" height="38" viewBox="0 0 200 200">
                    <defs>
                        <linearGradient id="sg" x1="0%" y1="0%" x2="100%" y2="100
                            <stop offset="0%" style="stop-color:#bf00ff"/>
                            <stop offset="100%" style="stop-color:#7b2fff"/>
                        </linearGradient>

```

```

        </defs>
        <path d="M100,100 C100,60 140,40 160,70 C180,100 160,140 130,
            C90,165 50,140 40,100 C25,50 60,15 100,15
            C150,15 185,55 185,100 C185,155 140,190 90,185"
            fill="none" stroke="url(#sg)" stroke-width="18"
            stroke-linecap="round"/>

        </svg>
    </div>
    <p style='font-size:1rem; color:#ffffff; margin:8px 0 0 0; font-weight:
    </div>
    """ , unsafe_allow_html=True)

# Theme toggle
mode_label = "☀️ Light Mode" if st.session_state.dark_mode else "🌙 Dark Mode"
if st.button(mode_label, use_container_width=True):
    st.session_state.dark_mode = not st.session_state.dark_mode
    st.rerun()

# Name badge
st.markdown(f"""
    <div class='name-badge'>👤 {st.session_state.user_name}</div>
    """, unsafe_allow_html=True)

if st.button("✦ New Chat", use_container_width=True):
    st.session_state.messages = []
    st.session_state.pending_image = None
    st.rerun()

if st.button("🔧 Change Name", use_container_width=True):
    st.session_state.user_name = None
    st.session_state.messages = []
    st.session_state.memory_facts = []
    st.rerun()

# Memory display
if st.session_state.memory_facts:
    st.markdown("<br>", unsafe_allow_html=True)
    st.markdown("<p class='history-label'>🧠 WHAT I REMEMBER</p>", unsafe_all
    for fact in st.session_state.memory_facts[:5]:
        st.markdown(f"<span class='memory-pill'>• {fact[:40]}</span>", unsafe

st.markdown("<br>", unsafe_allow_html=True)

# Grouped chat history
today, yesterday, this_week, older = group_histories_by_date(st.session_state

def render_history_section(label, chats, offset=0):

```

```

        if chats:
            st.markdown(f"<p class='history-label'>{label}</p>", unsafe_allow_html=True)
            for i, chat in enumerate(chats[:5]):
                if st.button(f"↩ {chat['title']}", key=f"hist_{label}_{offset}_{i}"):
                    st.session_state.messages = chat["messages"]
                    st.rerun()

    if not st.session_state.chat_histories:
        st.markdown("<p style='color:#4b5563; font-size:0.8rem;'>No conversations")
    else:
        render_history_section("TODAY", today, 0)
        render_history_section("YESTERDAY", yesterday, 10)
        render_history_section("THIS WEEK", this_week, 20)
        render_history_section("OLDER", older, 30)

# Model badge
st.markdown("""
<div style='margin-top:12px; padding:8px 12px; background:rgba(0,212,255,0.2); border:1px solid rgba(0,212,255,0.2); border-radius:10px;'>
<p style='color:#00d4ff; font-size:0.7rem; font-weight:600; margin:0'>
<p style='color:#e5e7eb; font-size:0.75rem; margin:0;'>Llama 4 Scout
<p style='color:#6b7280; font-size:0.65rem; margin:2px 0 0 0;'>Vision
</div>
""", unsafe_allow_html=True)

st.markdown("""
<div class='about-section'>
<p style='color:#9b59ff; font-size:0.82rem; font-weight:600; margin:0'>
<p style='color:#9ca3af; font-size:0.75rem; line-height:1.5; margin:0'>
Next-generation AI architecture delivering high-throughput autonomous
</p>
<p style='color:#6b7280; font-size:0.7rem; font-weight:600; margin:0'>
<span class='founder-tag'>🚀 Samuel Frimpong</span>
<span class='founder-tag'>💡 Cardinal Kofi Nsiah</span>
<span class='founder-tag'>⚡ Gabriel Ahwireng</span>
<span class='founder-tag'>🌀 Gideon Ahwireng</span>
<p style='color:#4b5563; font-size:0.65rem; text-align:center; margin:0'>
</div>
""", unsafe_allow_html=True)

# =====
# WELCOME DASHBOARD
# =====

if not st.session_state.messages:
    greeting = get_time_greeting()
    st.markdown(f"""

```

```

<div style='padding: 40px 0 20px 0;*>
  <div class='spiral-idle' style='display: block; margin-bottom: 15px;*>
    <svg width="55" height="55" viewBox="0 0 200 200">
      <defs>
        <linearGradient id="spiralMain2" x1="0%" y1="0%" x2="100%
          <stop offset="0%" style="stop-color: #bf00ff"/>
          <stop offset="100%" style="stop-color: #7b2fff"/>
        </linearGradient>
      </defs>
      <path d="M100,100 C100,60 140,40 160,70 C180,100 160,140 130,
        C90,165 50,140 40,100 C25,50 60,15 100,15
        C150,15 185,55 185,100 C185,155 140,190 90,185"
        fill="none" stroke="url(#spiralMain2)" stroke-width="18
        stroke-linecap="round"/>
    </svg>
  </div>
  <p class='greeting'>{greeting}</p>
  <p class='greeting-sub'>{st.session_state.user_name} 🙌</p>
  <p style='color: #9ca3af; font-size: 0.95rem; margin-top: 0;*>How can I
</div>
""" , unsafe_allow_html=True)

suggestions = [
  ("🧠", "Help me design a website for my business"),
  ("📈", "Explain forex trading strategies"),
  ("🤖", "What can AI automation do for my company"),
  ("✍️", "Write an executive pitch proposal for me"),
]

col1, col2 = st.columns(2)
for idx, (icon, text) in enumerate(suggestions):
  with col1 if idx % 2 == 0 else col2:
    if st.button(f"{icon} {text}", key=f"suggest_{idx}", use_container_w
      st.session_state.messages.append({"role": "user", "content": text
      st.rerun()

# =====
# RENDER CHAT HISTORY
# =====
for i, msg in enumerate(st.session_state.messages):
  if msg["role"] == "user":
    with st.chat_message("user"):
      if isinstance(msg.get("content"), list):
        for block in msg["content"]:
          if block["type"] == "text":
            st.write(block["text"])
          elif block["type"] == "image url":

```

```

        st.image(base64.b64decode(block["image_url"]["url"].split
else:
    st.write(msg["content"])
else:
    with st.chat_message("assistant"):
        st.write(msg["content"])
        # Copy button for each response
        msg_id = f"msg_{i}"
        st.markdown(f"""
            <button class='copy-btn' onclick='
                navigator.clipboard.writeText({json.dumps(msg["content"])});
                this.innerText="✅ Copied!";
                setTimeout(=>this.innerText="📋 Copy", 1500);
            '>📋 Copy</button>
            """, unsafe_allow_html=True)

# =====
# IMAGE UPLOAD ZONE
# =====
st.markdown("<div class='upload-zone'>", unsafe_allow_html=True)
st.markdown("<span class='upload-label'>🖼️ Attach an image for VectorWare to anal
uploaded_file = st.file_uploader(
    label="upload",
    type=["png", "jpg", "jpeg", "gif", "webp"],
    label_visibility="collapsed",
    key="image_uploader"
)
st.markdown("</div>", unsafe_allow_html=True)

if uploaded_file is not None:
    b64, mime = encode_image(uploaded_file)
    st.session_state.pending_image = {"base64": b64, "mime_type": mime, "name": u
    st.markdown(f"<div class='img-preview-pill'>🖼️ {uploaded_file.name} attached<

# =====
# CONTROL BAR - MIC + INPUT
# =====
col_input, col_mic = st.columns([0.88, 0.12])

with col_mic:
    st.markdown("<div style='padding-top:5px;'>", unsafe_allow_html=True)
    try:
        from streamlit_mic_recorder import mic_recorder
        audio = mic_recorder(
            start_prompt="🎙️",

```

```

        stop_prompt="🛑",
        just_once=True,
        use_container_width=True,
        key="mic"
    )
    if audio:
        import io
        audio_bytes = io.BytesIO(audio["bytes"])
        audio_bytes.name = "audio.wav"
        with st.spinner(""):
            transcription = client.audio.transcriptions.create(
                model="whisper-large-v3",
                file=audio_bytes,
            )
            if transcription.text:
                st.session_state.voice_prompt = transcription.text
                st.rerun()
    except Exception:
        st.markdown("🔊", unsafe_allow_html=True)
        st.markdown("</div>", unsafe_allow_html=True)

with col_input:
    prompt = st.chat_input(f"Ask VectorWare anything, {st.session_state.user_name}")

# Pick up voice input
if st.session_state.voice_prompt:
    prompt = st.session_state.voice_prompt
    st.session_state.voice_prompt = None

# =====
# MESSAGE PROCESSING
# =====
if prompt:
    # Build user message
    if st.session_state.pending_image:
        img = st.session_state.pending_image
        user_message_content = [
            {"type": "text", "text": prompt},
            {"type": "image_url", "image_url": {"url": f"data:{img['mime_type']}"}},
        ]
    with st.chat_message("user"):
        st.write(prompt)
        st.image(base64.b64decode(img["base64"]), width=260)
        st.session_state.messages.append({"role": "user", "content": user_message_content})
        st.session_state.pending_image = None
    else:

```

```

    st.session_state.messages.append({"role": "user", "content": prompt})
    st.chat_message("user").write(prompt)

# Extract and save memory facts
new_facts = extract_facts(prompt, st.session_state.user_name)
if new_facts:
    for fact in new_facts:
        if fact not in st.session_state.memory_facts:
            st.session_state.memory_facts.append(fact)
    save_memory(st.session_state.user_name, st.session_state.memory_facts)

# Check if web search needed
search_context = ""
search_used = False
if needs_web_search(prompt):
    with st.spinner("🌐 Searching the web..."):
        search_result = web_search(prompt)
        if search_result:
            search_context = f"\n\nLive web search results:\n{search_result}"
            search_used = True

# Show thinking process
thinking_placeholder = st.empty()
thinking_steps = []
if search_used:
    thinking_steps.append("🌐 Searched the web for live data")
if new_facts:
    thinking_steps.append(f"🧠 Remembered: {' '.join(new_facts[:2])}")
if st.session_state.pending_image:
    thinking_steps.append("🖼️ Analyzing attached image")
thinking_steps.append("🌀 Generating response...")

thinking_html = "<div class='thinking-box'>" + " &nbsp;&nbsp;&nbsp;".join(thinking_placeholder.markdown(thinking_html, unsafe_allow_html=True))

if search_used:
    st.markdown("<span class='search-pill'>🌐 Answer includes live web data</")

# Thinking loader
thinking_loader = st.empty()
thinking_loader.markdown("""
<div style='display: flex; align-items: center; gap: 12px; padding: 8px 0;'>
  <div class='spiral-thinking'>
    <svg width="28" height="28" viewBox="0 0 200 200">
      <defs>
        <linearGradient id="st" x1="0%" y1="0%" x2="100%" y2="100"
          <stop offset="0%" style="stop-color: #bf00ff"/>

```



```

        <stop offset="100%" style="stop-color:#00d4ff"/>
    </linearGradient>
</defs>
<path d="M100,100 C100,60 140,40 160,70 C180,100 160,140 130,
        C90,165 50,140 40,100 C25,50 60,15 100,15
        C150,15 185,55 185,100 C185,155 140,190 90,185"
        fill="none" stroke="url(#st)" stroke-width="18"
        stroke-linecap="round"/>

    </svg>
</div>
<p style='color:#9b59ff; margin:0; font-size:0.9rem; font-weight:500;
</div>
""", unsafe_allow_html=True)

# Memory context
memory_context = ""
if st.session_state.memory_facts:
    memory_context = f"\n\nWhat you know about {st.session_state.user_name}:

# System prompt
system_prompt = f""You are VectorWare AI, a highly intelligent, professional
The user's name is {st.session_state.user_name}. Address them by name naturally w
Be concise, insightful, and professional. You can analyze images, search the web,

api_messages = [{"role": "system", "content": system_prompt}]
for m in st.session_state.messages:
    api_messages.append({"role": m["role"], "content": m["content"]})

stream = client.chat.completions.create(
    model="meta-llama/llama-4-scout-17b-16e-instruct",
    messages=api_messages,
    max_tokens=2048,
    stream=True
)

thinking_placeholder.empty()
thinking_loader.empty()

with st.chat_message("assistant"):
    response_placeholder = st.empty()
    full_response = ""
    for chunk in stream:
        content_chunk = chunk.choices[0].delta.content
        if content_chunk:
            for char in content_chunk:
                full_response += char
            response_placeholder.markdown(full_response + "▮")

```

```

        time.sleep(0.005)
    response_placeholder.markdown(full_response)

    # Copy button
    st.markdown(f"""
        <button class='copy-btn' onclick='
            navigator.clipboard.writeText({json.dumps(full_response)});
            this.innerText="✅ Copied!";
            setTimeout(=>this.innerText="📋 Copy", 1500);
        '>📋 Copy</button>
    """, unsafe_allow_html=True)

st.session_state.messages.append({"role": "assistant", "content": full_respon

# Save to history
if len(st.session_state.messages) == 2:
    title = prompt[:26] + "..." if len(prompt) > 26 else prompt
    st.session_state.chat_histories.insert(0, {
        "title": title,
        "messages": list(st.session_state.messages),
        "time": datetime.now().strftime("%H:%M"),
        "date": datetime.now()
    })
st.rerun()

```